

Gesture Recognition via Mediapipe and Custom Deep Learning Models

Alessandro Trusendi, Antonio Querci
Department of Information Engineering

July 14, 2025

Contents

Abstract	3
1 Introduction	4
2 System Overview and Architecture	5
2.1 Architecture Overview	5
2.2 Component Breakdown	5
2.2.1 Client: Flask Web Application	5
2.2.2 Gesture Recognition Engine	6
2.2.3 Server: Command Execution	6
2.2.4 Constants module	6
2.2.5 Main module	6
2.3 Branch Overview	7
3 Flask Client Application	9
3.1 Core Features	9
3.2 Global variables	9
3.3 Configuration Management	9
3.4 Root Route	10
3.5 Gesture Recognition Process Control	11
3.6 Live Webcam Stream	13
3.7 Real-Time Gesture Retrieval	14
3.8 Server Health and Cleanup	15
3.9 Summary	16
4 Branch Analysis: Pretrained Mediapipe	17
4.1 Overview	17
4.2 Integration with System	17
4.3 How It Works	19
4.3.1 Callback Structure	21
4.3.2 Webcam Troubleshooting	22
4.4 Advantages and Limitations	24
4.5 Conclusion	24
5 Branch Analysis: Custom DNN Model	25
5.1 Overview	25
5.2 Model Architecture	25
5.3 Data Preparation and Training	26
5.3.1 Model Evaluation Metrics	27
5.3.2 Precision–Recall Tradeoff Analysis	27
5.3.3 Confusion Matrix	29
5.3.4 Confusion Matrix Analysis	29
5.3.5 Training and Validation Curve Analysis	30
5.4 Advantages and Limitations	31
5.5 Comparison with Pretrained Mediapipe	32
5.6 Conclusion	32

6	Branch Analysis: Custom Mediapipe Model	33
6.1	Overview	33
6.2	Custom Dataset	33
6.3	Training Script	33
6.4	Conclusion	34
7	Client-Server Interaction	35
7.1	Architecture Overview	35
7.2	Client Side	35
7.3	Server Side	37
7.4	Conclusion	39
8	Frontend Interface	40
8.1	Overview	40
8.2	HTML Template	40
8.3	JavaScript Logic	41
8.4	Flask Routing Logic	41
8.5	User Workflow	41
8.6	Future UI Improvements	41
9	Docker and Development Container Integration	42
9.1	Motivation	42
9.2	Dockerfile Benefits	42
9.3	Dev Container Configuration	42
10	Discussion and Future Work	44
10.1	Summary of Achievements	44
10.2	Technical Challenges	44
10.3	Strengths of the System	44
10.4	Future Work	44

Abstract

We present a gesture recognition system that enables human-computer interaction using hand movements captured via webcam. The system was developed in three variants: one using a pretrained Mediapipe model and, on a custom dataset, one with a custom deep neural network (DNN) and one with a Mediapipe-based architecture. The application follows a client-server architecture: the client is a Flask web application with an AJAX-based interface, and the server executes OS-level commands upon user gesture input. Gesture recognition is handled in a separate Python process. We use a separate process to send commands to the Windows Python server too. For the custom deep neural network version, we evaluate the model performance, accuracy (for the other two variants, we could not provide these statistics for reasons discussed below).

1 Introduction

In this project, we designed and implemented a gesture-controlled system capable of executing operating system commands based on hand signals captured via webcam. The architecture is modular and supports multiple gesture recognition engines, each implemented in separate Git branches:

- A pretrained Mediapipe model for rapid development and robustness.
- A custom deep learning model trained from scratch on a curated dataset.
- A hybrid approach using Mediapipe’s architecture with a custom-trained model.

The system architecture separates the user interface (a Flask web client) from gesture recognition and command execution, allowing asynchronous and distributed operation. The user interacts with a simple web UI, which triggers the backend recognizer to process video input and classify gestures.

This report provides a detailed technical analysis of each model variant, implementation strategy, and integration pipeline. We also propose future improvements.

2 System Overview and Architecture

2.1 Architecture Overview

The system follows a client-server paradigm. The client is a Flask-based web application that allows the user to trigger gesture recognition through a browser interface. Upon user request, a background Python process is launched which captures webcam video, classifies hand gestures using one of the three model variants, and returns an interpreted command. This command is then forwarded to the server, which executes it using Windows shell commands.

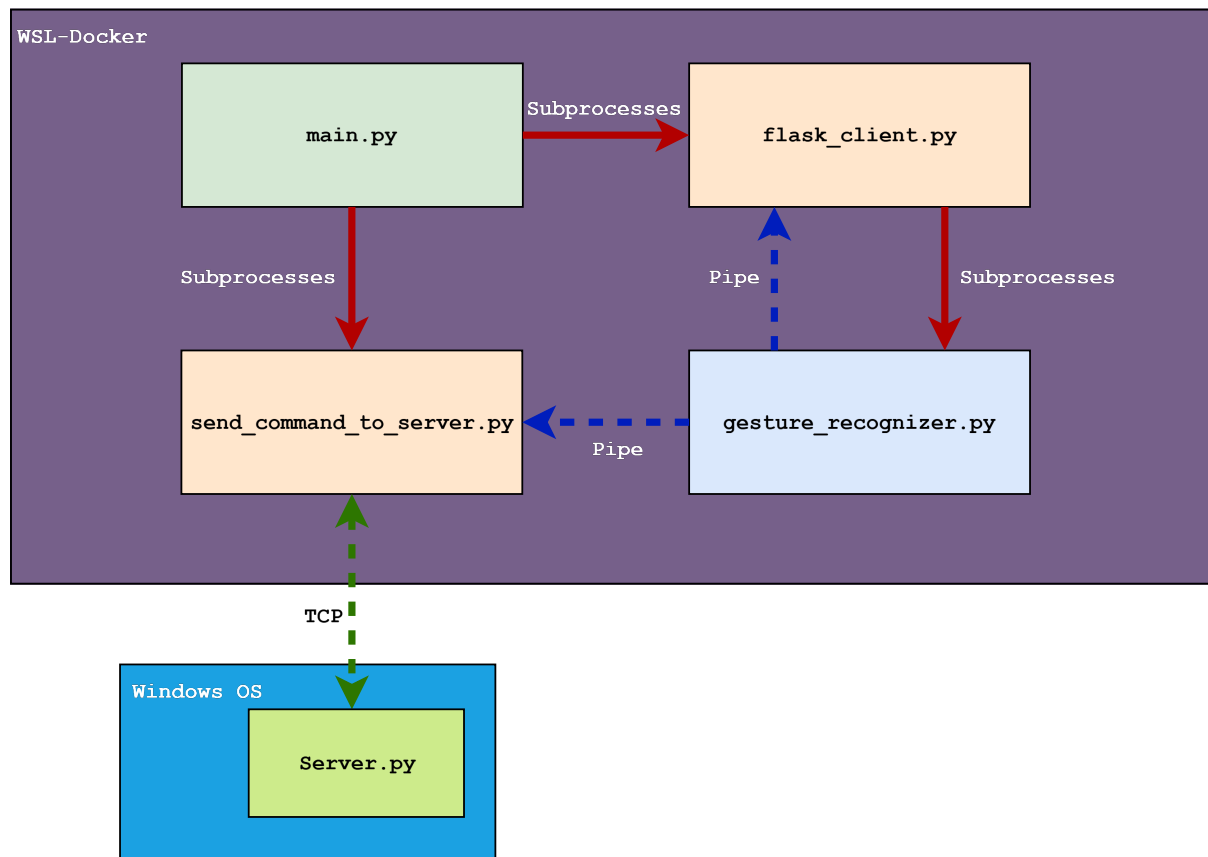


Figure 2.1: System Architecture Diagram

2.2 Component Breakdown

2.2.1 Client: Flask Web Application

The web client, implemented in `client/flask_client.py`, provides a simple interface with buttons or actions mapped to gestures. It uses AJAX calls to asynchronously send commands to the server and control the gesture recognition process.

- Launches the gesture recognizer via subprocess.
- Uses JavaScript and AJAX for communication.
- Let the user select (save and, later, load) the gesture to command associations.
- Displays feedback and recognition status.
- Let the user stop the client on command.

2.2.2 Gesture Recognition Engine

Located in `gesture_recognizer.py`, this engine runs in its own Python process. It handles:

- Webcam video stream capture
- Hand landmark extraction (via Mediapipe or custom pipeline)
- Gesture classification using the active model
- Sending the associated command to the captured gesture to the Flask process.

This component changes based on the pulled branch.

2.2.3 Server: Command Execution

The server, running `server.py`, receives gesture-based commands from the client via TCP communication and executes the corresponding Windows command on the machine that the server is hosted on.

- Ensures security by sanitizing inputs
- Executes mapped commands (e.g., raise volume, screenshot)
- Returns status updates to the client

2.2.4 Constants module

This module contains a `COMMANDS` tuple, which is used in `flask_client.py` and `gesture_recognizer.py`. In the `custom_dnn` branch, this module also contains `GESTURE` tuple, which is used in the same modules as `COMMANDS`. `GESTURE` tuple varies based on the gestures upon which the model is trained (in our case, `custom_dnn` and `custom_mediapipe_model` use the same `GESTURE` tuple, while MediaPipe pretrained model - which is used in `main` branch - uses another `GESTURE` tuple).

2.2.5 Main module

This is the entrypoint for the client application:

- Sets the multiprocessing start method to 'spawn' for clean child process creation.
- Initializes a multiprocessing queue for communication between client and server.
- Starts a separate process to listen to the queue and send commands to the server.
- Attaches the queue to the Flask client for global access.
- Runs the Flask application to handle incoming HTTP requests.
- On Flask shutdown, signals the command-sending process to terminate and waits for it to finish.

Listing 2.1: main.py

```
def main():
    # Set the start method for multiprocessing to 'spawn'
    multiprocessing.set_start_method("spawn", force=True)

    # Initialize a single multiprocessing queue for communication
    ↪ between gesture_recognizer.py and
    ↪ send_command_to_server.py
    gesture_recognizer_to_socket_queue = multiprocessing.Queue()
    flask_client.gesture_recognizer_to_socket_queue =
    ↪ gesture_recognizer_to_socket_queue

    # Initialize a single multiprocessing queue for communication
    ↪ between Flask client and send_command_to_server.py
    server_is_running = multiprocessing.Value(ctypes.c_bool,
    ↪ False) # Shared boolean
    flask_client.server_is_running = server_is_running

    # Start a separate process to listen to the queue and send
    ↪ commands to the server
    send_proc = multiprocessing.Process(
        target=send_command_to_server,
        args=(gesture_recognizer_to_socket_queue,
        ↪ server_is_running,)
    )
    send_proc.start()

    # Start Flask (this blocks until you stop it with CTRL-C)
    flask_client.app.run(host="0.0.0.0", port=8080, threaded=True
    ↪ )

    # When Flask stops, signal the command-sending process to
    ↪ terminate
    send_proc.terminate()
    send_proc.join()

if __name__ == "__main__":
    main()
```

In order to start the client from the project root folder and correctly set PYTHONPATH variable, we created the *start_client.sh* shell script.

2.3 Branch Overview

Each branch in the repository supports a different model implementation for gesture recognition:

- **main:** Uses Google's pretrained Mediapipe hand landmark detection and classification.
- **custom_dnn:** A custom convolutional neural network trained on a custom dataset (defined in a Jupyter notebook).

- **custom_mediapipe_model:** Custom dataset + training, but uses Mediapipe's internal architecture and tools to convert it into a usable pipeline.

This modular design allows testing of various model strategies without changing the overall system architecture.

3 Flask Client Application

The `flask_client.py` module implements the web interface and orchestration logic for the gesture recognition client. It uses the Flask framework to serve HTML pages, manage gesture-command configurations, launch gesture recognition processes, and stream webcam video to the browser in real-time.

3.1 Core Features

- Launches a gesture recognition subprocess using Python’s `multiprocessing` module.
- Exposes HTTP endpoints for controlling recognition start/stop actions. Each HTTP response is sent as a JSON file.
- Allows users to configure and map gestures (e.g., `Thumb_Up`, `Victory`) to system commands.
- Saves and loads configurations via JSON files.
- Streams live video using MJPEG and OpenCV.
- Shares memory between processes to expose the last recognized gesture.

3.2 Global variables

In `flask_client.py`, global variables are used to maintain information needed by more functions (i.e, to enable communication between processes in the Flask-based gesture recognition system). The global variables are:

- `gesture_to_command`: A dictionary storing gesture-to-command mappings, modifiable by the user and used by the recognizer.
- `gesture_recognizer_to_socket_queue`: A multiprocessing queue for sending recognized gestures to the server for command execution.
- `server_is_running`: A shared memory flag indicating the server’s availability.
- `last_gesture`: A shared memory array updated by the recognizer and read by the web UI to show the current gesture.
- `CONFIG_DIR`: Path to saved configuration files, used across routes.

These are declared as globals to allow persistent, cross-request, and cross-process access—necessary due to Flask’s request-driven model and multiprocessing-based architecture.

3.3 Configuration Management

Users can assign system commands (like `open calculator` or `volume control`) to each supported gesture. The app provides:

- A dropdown for selecting existing configurations.

- An input field for creating new configuration files.
- Buttons to apply mappings temporarily or save them persistently as JSON.

The configuration is stored under `client/static/configs/` and validated using simple regex to prevent unsafe file names.

3.4 Root Route

The root route (`"/`) in `flask_client.py` serves as the main entry point for the web interface. It handles both GET and POST requests. When accessed via GET, it renders the configuration page, displaying available gestures, system commands, and any saved configurations. When accessed via POST, it processes user-submitted forms to either apply gesture-to-command mappings temporarily or save them permanently to a configuration file. These mappings allow the user to associate specific hand gestures with system commands. The route ensures that configurations are properly validated and saved in a consistent format, enabling persistent customization of gesture behavior through a simple and intuitive web interface.

Listing 3.1: `root` from `flask_client.py`

```
@app.route("/", methods=["GET", "POST"])
def index():
    # List of available configuration files (without .json
    ↪ extension)
    config_files = [f[:-5] for f in os.listdir(CONFIG_DIR) if f.
    ↪ endswith(".json")]
    # List of available gestures
    GESTURES = ("Thumb_Up", "Thumb_Down", "Open_Palm", "
    ↪ Closed_Fist", "Victory", "ILoveYou", "Pointing_Up")
    if request.method == "POST":
        global gesture_to_command
        # Used only for the "save" action
        # request.form.get("config_name_select") is used when the
        ↪ user selects a config from the dropdown
        # request.form.get("config_name_text") is used when the
        ↪ user types a new config
        # If both are empty, selected_config will be an empty
        ↪ string
        selected_config = request.form.get("config_name_select")
        ↪ or request.form.get("config_name_text", "")
        action = request.form.get("action")
        if action == "apply":
            # Update gesture_to_command but does not save to file
            for gesture in GESTURES:
                # We receive the gesture name as a string from
                ↪ the form, which is a FormData JavaScript
                ↪ object
                command = request.form.get(gesture)
                if command:
                    gesture_to_command[gesture] = command
                elif gesture in gesture_to_command:
                    del gesture_to_command[gesture]
            return jsonify({"status": "ok", "message": "
            ↪ Configuration applied successfully."})
```

```

elif action == "save" and is_valid_config_name(
    ↪ selected_config):
    print("[INFO] Valid configuration name")
    # Update gesture_to_command and saves configuration
    ↪ in a file
    for gesture in GESTURES:
        command = request.form.get(gesture)
        gesture_to_command[gesture] = command
    if selected_config:
        # Save the current gesture_to_command mapping to
        ↪ a JSON file
        path = os.path.join(CONFIG_DIR, selected_config +
            ↪ ".json")
        with open(path, "w") as f:
            json.dump(gesture_to_command, f, indent=2)
        return jsonify({"status": "ok", "message": "
            ↪ Configuration saved and applied
            ↪ successfully."})
    else:
        return jsonify({"status": "error", "message": "No
            ↪ selected configuration."}, 400)
else:
    return jsonify({"status": "error", "message": "
        ↪ Unknown action."}, 400)

# GET: only when the user opens the page for the first time
↪ or refreshes it
global recognition_active
return render_template(
    "index.html",
    gestures=GESTURES,
    commands=COMMANDS,
    active=recognition_active,
    configs=config_files,
)

```

3.5 Gesture Recognition Process Control

Flask routes `/start` and `/stop` control the background recognition process. When started, it initializes shared memory structures:

- A shared array `last_gesture` for writing recognized gestures.
- Queues for passing webcam frames or inter-process signals.

Recognition is handled by a subprocess that runs the function `start_gesture_recognition()`.

Listing 3.2: start recognition from `flask_client.py`

```

@app.route("/start", methods=["GET"])
def start_recognition() -> "Response":
    global recognition_active, recognition_process

    if not recognition_active:
        recognition_active = True
        # Initialize queues for inter-process communication

```

```

global webcam_frame_queue
webcam_frame_queue = multiprocessing.Queue()
global last_gesture
# 11 is the max string length in GESTURES list. +1 for \0
last_gesture = multiprocessing.Array(ctypes.c_char, 11+1)
# Pass gesture_to_command as an argument
global gesture_to_command
global gesture_recognizer_to_socket_queue
recognition_process = multiprocessing.Process(
    target=start_gesture_recognition,
    args=(gesture_to_command, webcam_frame_queue,
        ↪ gesture_recognizer_to_socket_queue,
        ↪ last_gesture,),
)
recognition_process.start()
return jsonify({"status": "ok", "active": True})

```

Mapped to the `"/start"` route with GET requests, This route **starts the gesture recognition process** as a background subprocess.

- Checks if recognition is already running using the `recognition_active` flag.
- If not running:
 - Sets `recognition_active = True`.
 - Initializes the **webcam frame queue** (`webcam_frame_queue`) to transfer frames.
 - Initializes the **shared memory array** (`last_gesture`) to store the last recognized gesture.
 - Allocates 12 bytes (for the longest gesture string + null terminator).
 - Starts a new subprocess using `multiprocessing.Process`, targeting `start_gesture_recognition()`.
 - * Passes arguments: gesture-command mapping, webcam frame queue, command queue, and shared gesture memory.
- **Prints a status message** and returns a JSON response with `"active": True`.

Listing 3.3: stop recognition from `flask_client.py`

```

@app.route("/stop", methods=["GET"])
def stop_recognition() -> "Response":
    global recognition_active, recognition_process

    # Check if recognition is already inactive
    if recognition_active is False:
        return jsonify({"status": "no", "active": False})
    recognition_active = False

    # If the recognition process is still running, terminate it
    ↪ and its associated queues with flask_client.py
    if recognition_process and recognition_process.is_alive():

```

```

        recognition_process.terminate()
        global webcam_frame_queue
        webcam_frame_queue.close()
        webcam_frame_queue.join_thread()
        recognition_process = None

    return jsonify({"status": "ok", "active": False})

```

Mapped to the `/stop` route with GET requests, this route **stops the gesture recognition process** and cleans up resources.

- Checks if recognition is active. If not, returns a response: `"active": False`.
- If active:
 - Sets `recognition_active = False`.
 - If the subprocess exists and is still running:
 - * Calls `terminate()` to kill the process.
 - * Closes and joins the webcam frame queue to release resources.
 - * Sets `recognition_process = None`.
- Prints status messages and returns a JSON response with `"active": False`.

3.6 Live Webcam Stream

The route `/video_feed` streams video frames from the webcam to the frontend using an MJPEG stream. Each frame is read from the recognition queue, encoded with OpenCV, and pushed to the browser using Flask's `Response` object with `multipart/x-mixed-replace`.

Listing 3.4: video feed from flask_client.py

```

@app.route("/video_feed", methods=["GET"])
def video_feed() -> "Response":
    def generate():
        while recognition_active:
            frame = webcam_frame_queue.get() if
                ↳ webcam_frame_queue else None
            if frame is None:
                continue

            if not ret:
                continue

            yield (b"--frame\r\n"
                   b"Content-Type: image/jpeg\r\n\r\n" + buffer.
                   ↳ tobytes() + b"\r\n")

    return Response(generate(), mimetype="multipart/x-mixed-
        ↳ replace; boundary=frame")

```

1. While the recognition is active a new frame is extracted from the shared queue.
 - (a) if its None we don't do anything

2. The frame is encoded as a JPEG (since it was a MJPG inside the queue)
3. The frame is converted in bytes and sent as a binary string to the Web UI which displays it as a video feed.

Listing 3.5: HTML panel for video feed

```
<div class="right-panel">
  <h2>Webcam Live</h2>
  <div class="webcam-container">
    
  </div>
  ...
</div>
```

3.7 Real-Time Gesture Retrieval

The route `/get_recognized_gesture` reads the most recent gesture from shared memory and sends it to the frontend asynchronously. This enables real-time updates in the user interface.

Listing 3.6: `get_recognized_gesture` from `flask_client.py`

```
@app.route("/get_recognized_gesture", methods=["GET"])
def send_recognized_gesture() -> "Response":
    global recognition_active
    if recognition_active is False:
        return jsonify({"status": "error", "message": "Gesture_
          ↪ recognizer_process_is_not_running."}), 503
    global last_gesture
    # Read string from shared memory
    raw_bytes = bytes(last_gesture[:]).rstrip(b'\x00') # Removes
      ↪ ending zeros
    gesture = raw_bytes.decode()
    return jsonify({"status": "ok", "message": gesture})
```

- **Checks if the gesture recognition process is active:**
 - If `recognition_active` is `False`, returns a 503 Service Unavailable error.
 - It is pointless to send recognized gesture if the webcam is not recording.
- **Accesses shared memory (`last_gesture`):**
 - This is a `multiprocessing.Array` shared between the Flask app and the gesture recognition subprocess.
 - `last_gesture` is a global variable because it needs to be accessed by multiple functions (`start_recognition()` and `send_recognized_gesture()`).
 - It holds a fixed-length byte array (the length is equal to the longest gesture name) containing the most recent gesture label.
- **Reads and decodes the gesture:**

- Converts the byte array to a Python `bytes` object.
- Uses `.rstrip(b'\x00')` to remove any null terminators from unused buffer space.
- Decodes the result into a UTF-8 string representing the gesture (e.g., "Thumb_Up").
- **Logs and returns the gesture:**
 - Prints the recognized gesture to the server console.
 - Sends it back to the browser as a JSON response in the format: "status": "ok", "message": "Gesture"

3.8 Server Health and Cleanup

Additional endpoints include:

- `/check_server` – to verify server availability.
- `/stop_client` – to shut down the client cleanly via SIGINT.
- `/get_json_file` – to dynamically load existing configuration mappings.

Listing 3.7: `check_server()` from `flask_client.py`

```
@app.route("/check_server", methods=["GET"])
def check_server() -> "Response":
    global server_is_running
    if server_is_running.value:
        return jsonify({"status": "ok", "message": "Connection_
            ↳ established."})
    else:
        return jsonify({"status": "error", "message": "Server_is_
            ↳ not_running."}), 503
```

This route is called every 5 seconds by `check_if_server_is_running` async function in `script.js`. It checks if the server is running and updates the flask interface accordingly depending on the boolean `server_is_running`, which is a shared `multiprocessing.Value` boolean between `flask_client.py` and `send_command_to_server.py`; this boolean is created on the client start by `main.py` as is updated by `send_command_to_server.py` process each time it begins or ends a connection with the Windows server.

Listing 3.8: `stop_client()` from `flask_client.py`

```
@app.route("/stop_client", methods=["GET"])
def stop_client() -> "Response":
    global recognition_active
    if recognition_active:
        stop_recognition()
    os.kill(os.getpid(), signal.SIGINT)
    return jsonify({"status": "ok", "message": "Client_stopped_
        ↳ successfully."})
```

This route is used to terminate the client process gracefully (`gesture_recognizer.py` module is coded to handle SIGTERM signal). It stops the gesture recognition process (if it is active) and returns a JSON response indicating success.

Listing 3.9: *get_json_file()* from flask_client.py

```
@app.route("/get_json_file", methods=["GET"])
def get_json_file() -> "Response":
    file_name = request.args.get("file") + ".json"
    if not file_name:
        return jsonify({"error": "File_name_not_provided"}), 400
    # Ensure the file is in the correct directory and has a .json
    ↪ extension
    if not os.path.isfile(os.path.join(CONFIG_DIR, file_name)):
        return jsonify({"error": "File_not_found"}), 404

    with open(os.path.join(CONFIG_DIR, file_name), "r") as f:
        data = json.load(f)

    return jsonify(data)
```

This route is used to load configuration files dynamically based on the request parameter 'file' (the Web UI provides a HTML select element which contains all saved configurations names. When the user selects a configuration name, an HTTP request - via AJAX - is sent to this route to get the JSON file associated with this name; that JSON file contains the gesture to commands associations). It ensures that the requested file exists and is a valid JSON file before serving it.

3.9 Summary

The flask_client.py file acts as the central hub for user interaction, process control, and inter-process communication. It connects gesture recognition logic with a web-based interface, enabling a seamless end-to-end experience for configuring, testing, and executing gesture-driven commands.

4 Branch Analysis: Pretrained Mediapipe

4.1 Overview

The main branch of the project utilizes Google's Mediapipe framework with its pretrained hand tracking and gesture classification modules. This approach is beneficial for rapid development, providing robust, low-latency hand landmark detection out-of-the-box.

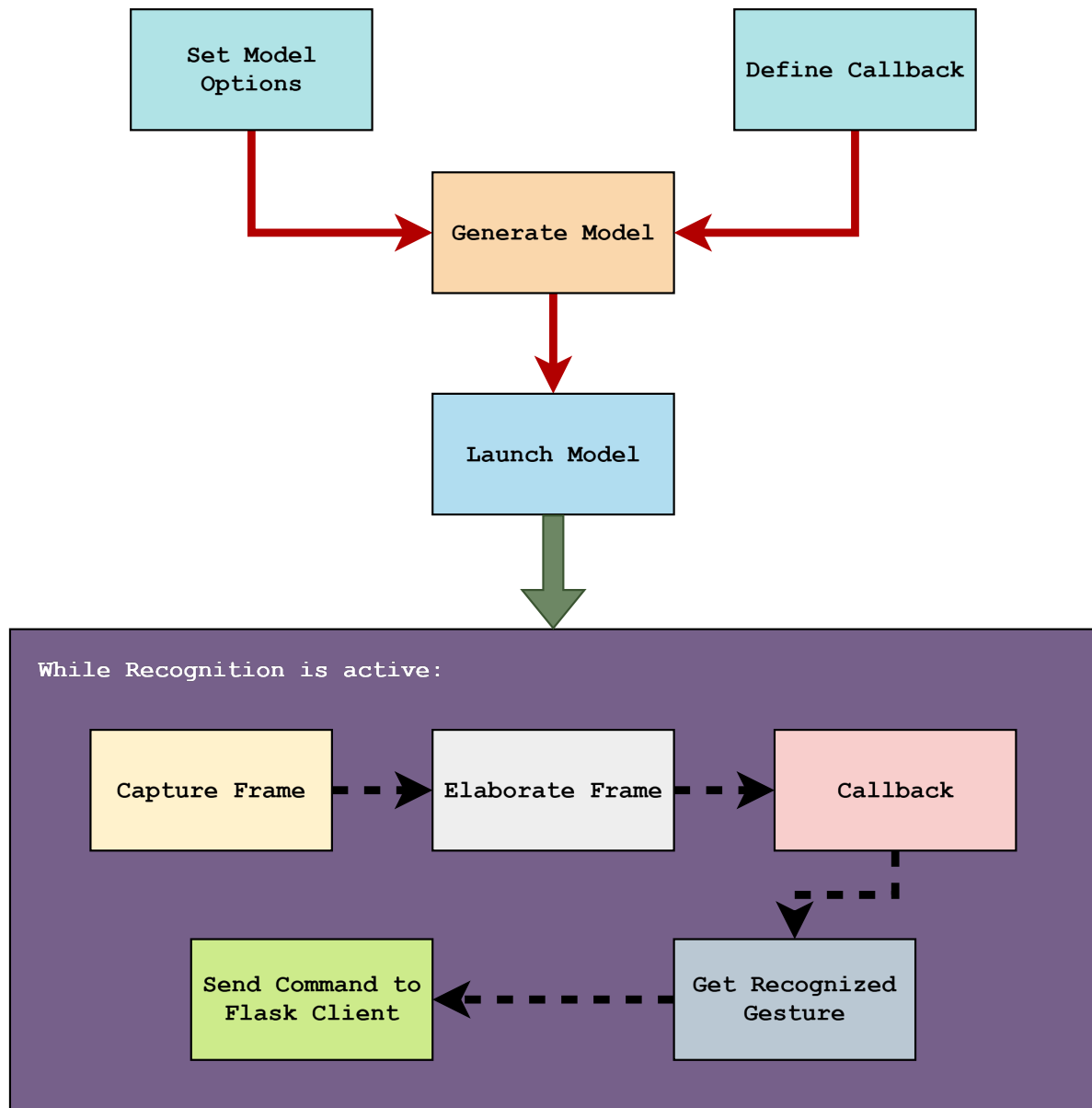


Figure 4.1: Placeholder: Mediapipe Graph Pipeline for Hand Recognition

4.2 Integration with System

This module is always started as a process is from the Flask client via the *start_recognition()* function. In the previous chapter, we explained how the recognition is started from the point of view of the Flask module. Here, instead, we are going to explain how the gesture recognition is started from the point of view of the `gesture_recognizer.py` module:

Listing 4.1: Launching recognizer from flask_client.py

```
@app.route("/start", methods=["GET"])
def start_recognition() -> "Response":

    global recognition_active, recognition_process

    if not recognition_active:
        recognition_active = True
        # Initialize queues for inter-process communication
        global webcam_frame_queue
        webcam_frame_queue = multiprocessing.Queue()
        global last_gesture
        # 11 is the max string length in GESTURES list. +1 for \0
        last_gesture = multiprocessing.Array(ctypes.c_char, 11+1)
        # Pass gesture_to_command as an argument
        global gesture_to_command
        global gesture_recognizer_to_socket_queue
        recognition_process = multiprocessing.Process(
            target=start_gesture_recognition,
            args=(gesture_to_command, webcam_frame_queue,
                ↪ gesture_recognizer_to_socket_queue,
                ↪ last_gesture,),
        )
        recognition_process.start()
    return jsonify({"status": "ok", "active": True})
```

We need to start this module as a process via the Flask client because it needs informations that will be decided by the app user at runtime:

- *gesture_to_command*: this dictionary will be used to get the command associated to the just processed webcam frame.
- *flask_to_web_interface_queue*: this *multiprocessing.Queue* will be used to display webcam feed in the web interface (the *gesture_recognizer* process will put here each captured webcam frame).
- *gesture_recognizer_to_socket_queue*: this *multiprocessing.Queue* will be used by *gesture_recognizer.py* to send commands in string format to the *send_command_to_server.py* process and eventually to the server on the Windows Host.
- *last_gesture*: this *multiprocessing.Array* of *ctypes.c_char* will be used to send the recognized gesture to the Flask client (the Flask client, then, will send this to the web interface so that the recognized gesture <p> can be updated).

In order to be started as a process, this whole module will be encapsulated in a function (*start_gesture_recognition*):

Listing 4.2: Launching recognizer from flask_client.py

```
def start_gesture_recognition(gesture_to_command: dict,
    ↪ webcam_queue: "multiprocessing.Queue",
    ↪ client_to_server_queue: "multiprocessing.Queue",
    ↪ last_gesture: "multiprocessing.Array") -> None:
```

4.3 How It Works

In this variant, the `gesture_recognizer.py` script initializes a gesture recognition process using the pretrained Mediapipe model. The initialization goes as follows:

Listing 4.3: Mediapipe pretrained model initialization pseudocode

```
import mediapipe as mp
import cv2

# Prepare the MediaPipe model path
model_path = os.path.join(os.path.dirname(__file__), "
    ↳ gesture_recognizer.task")

# Create aliases for classes to make code more readable
BaseOptions = mp.tasks.BaseOptions
GestureRecognizer = mp.tasks.vision.GestureRecognizer
GestureRecognizerOptions = mp.tasks.vision.
    ↳ GestureRecognizerOptions
GestureRecognizerResult = mp.tasks.vision.GestureRecognizerResult
VisionRunningMode = mp.tasks.vision.RunningMode

# Separate Mediapipe hands for drawing landmarks on frame
mp_hands = mp.solutions.hands
hands_draw = mp_hands.Hands(static_image_mode=False,
    ↳ max_num_hands=2)
mp_draw = mp.solutions.drawing_utils

# Create the GestureRecognizerOptions with the model path and
    ↳ result callback.
# The result callback is called every time a gesture is
    ↳ recognized.
# The running mode is set to LIVE_STREAM to process frames from
    ↳ the webcam.
# The number of hands is set to 2 to recognize gestures from both
    ↳ hands.
# The model asset path is set to the path of the gesture
    ↳ recognizer model file.
options = GestureRecognizerOptions(
    base_options=BaseOptions(model_asset_path=model_path),
    running_mode=VisionRunningMode.LIVE_STREAM,
    result_callback=get_result,
    num_hands=2
)

with GestureRecognizer.create_from_options(options) as recognizer
    ↳ :
    # Open webcam
    cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
    # Handle SIGTERM gracefully
    signal.signal(signal.SIGTERM, make_sigterm_handler(cap))
    # Webcam options
    cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
```

```

cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

if not cap.isOpened():
    return

try:
    while True:
        ret, frame = cap.read()
        if not ret:
            tm.sleep(0.1)
            continue
        # OpenCV captures webcam frames in BGR color
        # MediaPipe needs RGB frames, so we convert these.
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        # Convert webcam frame in a numpy array
        # This is needed by MediaPipe for correct image
        ↪ processing
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB,
        ↪ data=rgb_frame)
        # The frame timestamp is calculated in milliseconds.
        # This is used to synchronize the frames with the
        ↪ results.
        # The timestamp is calculated using the OpenCV tick
        ↪ count and tick frequency.
        frame_timestamp_ms = int(cv2.getTickCount() / cv2.
        ↪ getTickFrequency() * 1000)
        # Call the recognizer to process the image and
        ↪ recognize gestures.
        # The recognizer will call the 'get_result' function
        ↪ with the recognized gestures.
        recognizer.recognize_async(mp_image,
        ↪ frame_timestamp_ms)

        # Draw MediaPipe hands landmarks
        draw_results = hands_draw.process(rgb_frame)
        if draw_results.multi_hand_landmarks:
            for hand_landmarks in draw_results.
            ↪ multi_hand_landmarks:
                mp_draw.draw_landmarks(frame, hand_landmarks,
                ↪ mp_hands.HAND_CONNECTIONS)
        # Update web UI webcam feed
        webcam_queue.put(frame.copy())

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
finally:
    cap.release()
    tm.sleep(0.1)
    cv2.destroyAllWindows()

```

As we can see from the pseudocode above, the gesture recognition process works this way:

1. We create model options.

2. We set up hand landmarks drawing objects.
3. We initialize the model object with the previously created options.
4. We start the gesture recognition task
5. We handle SIGTERM to gracefully shutdown the webcam (when the client stops or when the user wants to stop the recognition).
6. We setup and start webcam feed (this may need troubleshooting. See below for additional help)
7. In a loop, we capture webcam frames, we process them and we asynchronously retrieve their associated gesture.
 - (a) The callback will get the recognized gesture and send the (eventually, it might be "None") associated command to the server.
8. We send to the Flask client task the current webcam frame (to update Flask UI webcam feed).

4.3.1 Callback Structure

Each time a frame is captured, the recognition model (called `gesture_recognizer.task`) processes it and, once it has finished, the recognizer will call the `get_result` function, where the prediction result becomes available.

Listing 4.4: Callback structure

```
import mediapipe as mp

def get_result(result: GestureRecognizerResult, output_image: mp.
    ↳ Image, timestamp_ms: int) -> None:

    def save_last_gesture(recognized_gesture) -> None:
        # Write the new gesture
        for i, char in enumerate(recognized_gesture):
            last_gesture[i] = char.encode() if isinstance(
                ↳ char, str) else char
        # Empty the remaining parts of the buffer
        for i in range(len(recognized_gesture), len(
            ↳ last_gesture)):
            last_gesture[i] = b'\x00'

    nonlocal counter
    counter += 1
    if counter % 10 != 0:
        return
    # Print all recognized category_names:
    for gesture_list in result.gestures:
        for classification in gesture_list:
            if classification.category_name is not None:
                # Extract the recognized gesture
```

```

        recognized_gesture = classification.
        ↪ category_name
    # Send the recognized gesture to the flask
    ↪ client:
    save_last_gesture(recognized_gesture)
    # If this gesture is not bound to any command
    ↪ , continue
    if gesture_to_command is None or not
    ↪ gesture_to_command:
        continue
    if gesture_to_command.get(recognized_gesture)
    ↪ is None:
        continue
    # Send the associated command to the
    ↪ send_command_to_server.py module:
    command = gesture_to_command.get(
    ↪ recognized_gesture)
    if command in COMMANDS:
        client_to_server_queue.put(command)
    # If there are no gestures recognized, send "None" to the
    ↪ Flask client
    if not result.gestures:
        save_last_gesture("None")

```

This function processes the gesture recognition result, sending recognized gesture commands to the server at specified intervals. It also saves the last gesture recognized into a shared memory buffer with *flask-client.py* to display it on the web interface.

- There are a lot of frames to process: in order not to overload the server, we process one command every tenth. We do not "reduce" the number of taken frames in the while loop because we have seen that it causes prediction issues and choppy webcam feed.
- The recognized gesture (even if "None", which means no gesture was recognized) is passed to the Flask client via a shared memory buffer, *last_gesture* (a *multiprocessing.Array* of *ctypes.c_char*), so as to display it in the web interface.
- If the recognized gesture is associated to a command, this command itself will be sent to the *send_command_to_server.py* process, which will communicate the command to the server via TCP.

4.3.2 Webcam Troubleshooting

First off, the webcam needs to be attached to wsl before the Docker container is ran, otherwise the webcam will not show up inside it. To attach the webcam to WSL, there are instructions inside the *README.md* file on the GitHub Repository and a *star-webcam.ps1* script to completely automate the process (you must launch this script from your Windows host). We used *usbipd* command to bind the webcam to WSL2 (we found out that newer WSL2 versions have webcam drivers installed in the kernel by default, so the binding is all we need).

Secondly, if your machine supports CUDA, it drastically improves gesture recognition speed and performance, making the web UI webcam feed smoother), so we advise you to install it on WSL2.

If you get an OpenCV error while running the client or you can't see the webcam feed, it might be that `gesture\_recognizer.py` is trying to capture frames with a resolution and a video format which are not supported by your webcam.

In the file `gesture_recognizer.py`, you will find the following lines of code:

Listing 4.5: Hand detection using Mediapipe

```
import cv2

# Select a webcam to capture video from.
cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
# Register the SIGTERM signal handler to release the webcam and
    ↪ close OpenCV windows.
signal.signal(signal.SIGTERM, make_sigterm_handler(cap))
# Set the video codec, frame width, and height.
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
# (Optional) Select fps
# cap.set(cv2.CAP_PROP_FPS, 30)
```

These lines set the video frame format and resolution for the webcam. If you select a format or resolution that is not supported by your specific webcam, OpenCV may fail to capture video or the webcam may not work at all.

If your webcam does not work out of the box, you should try changing the format and resolution in these lines to values that are supported by your device.

By default, we use the MJPEG format and a resolution of 640x480, which are commonly supported by most webcams.

4.4 Advantages and Limitations

Advantages

- No training required; model is already optimized and deployed.
- Extremely fast inference (real-time even on CPUs).
- High robustness in varying lighting and hand positions.

Limitations

- Fixed gesture vocabulary — you can't add new gestures.

4.5 Conclusion

The pretrained Mediapipe solution offers a stable and efficient baseline for gesture recognition. It is well-suited for prototypes and constrained gesture sets. However, its limitations in customization and scalability prompted the development of the custom DNN and custom Mediapipe model variants, which are analyzed in the next chapters.

5 Branch Analysis: Custom DNN Model

5.1 Overview

The `custom_dnn` branch introduces a deep learning-based approach for gesture recognition. This version was created with the intent of understanding how to create a DNN. Anyway, its architecture is (probably) way simpler than the MediaPipe one, so it has worse performances (less accuracy and it needs more resources to perform well).

The model is trained using *PyTorch* module and a curated dataset of hand landmark vectors, labeled with corresponding gestures extracted by MediaPipe. The training and evaluation are handled via a Jupyter notebook: `model_training.ipynb`. Once trained, the model is exported still using *PyTorch* and loaded into the runtime gesture recognition engine.

The model recognizes two hands, but can only associate commands to one hand. If you are performing different gestures with your hands, the priority hand is the right one.

5.2 Model Architecture

The CNN architecture is intentionally lightweight to support real-time inference on CPU environments. Input features are 3D hand landmarks ($21 \text{ points} \times 3 \text{ coordinates}$) for both hands, flattened into a 126-dimensional vector.

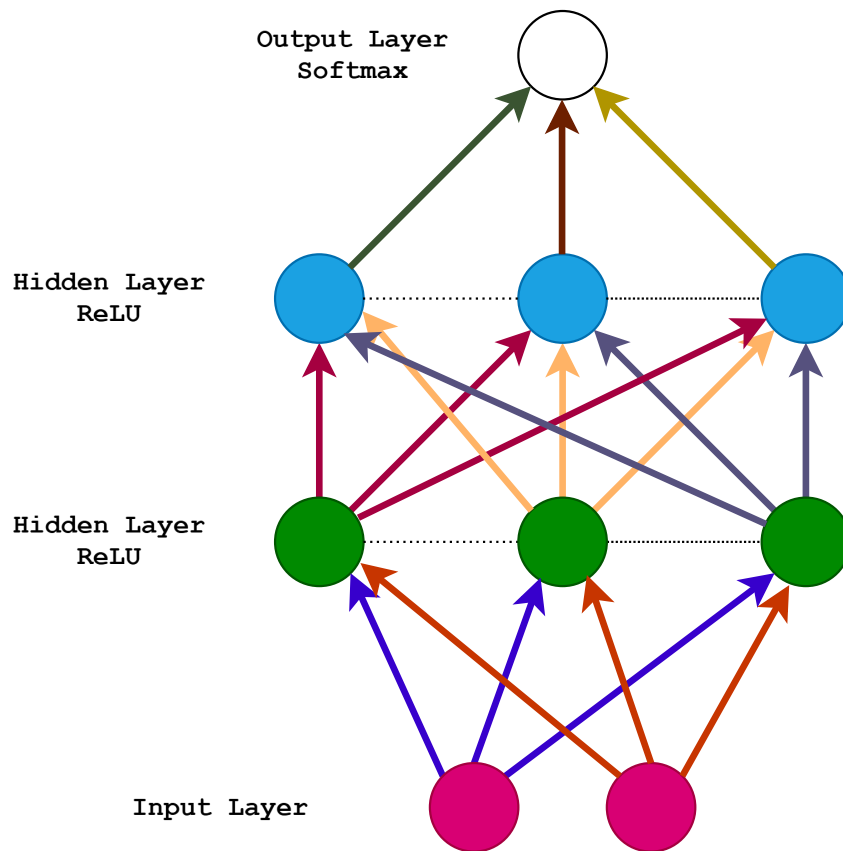


Figure 5.1: Custom DNN Architecture

Architecture Summary

- **Input:** 63*2 values (flattened hand landmarks)
- **Hidden Layer 1:** 128 neurons, ReLU activation
- **Dense Layer 1:** 128 neurons, ReLU activation
- **Output Layer:** Softmax over 18 + 1 gesture classes

5.3 Data Preparation and Training

The dataset consists of annotated hand landmark vectors extracted using Mediapipe and saved into a NPZ file. Each row contains a vector of 63 values (21 landmarks $\times$ X,Y,Z) followed by a gesture label. We used as our main dataset the Hagrid-30k-384p. Since this dataset did not include a "None" class (for unrecognized gestures), we downloaded it from the original Hagrid GitHub but, because there all the pictures were in Full HD, we downgraded those to 384p.

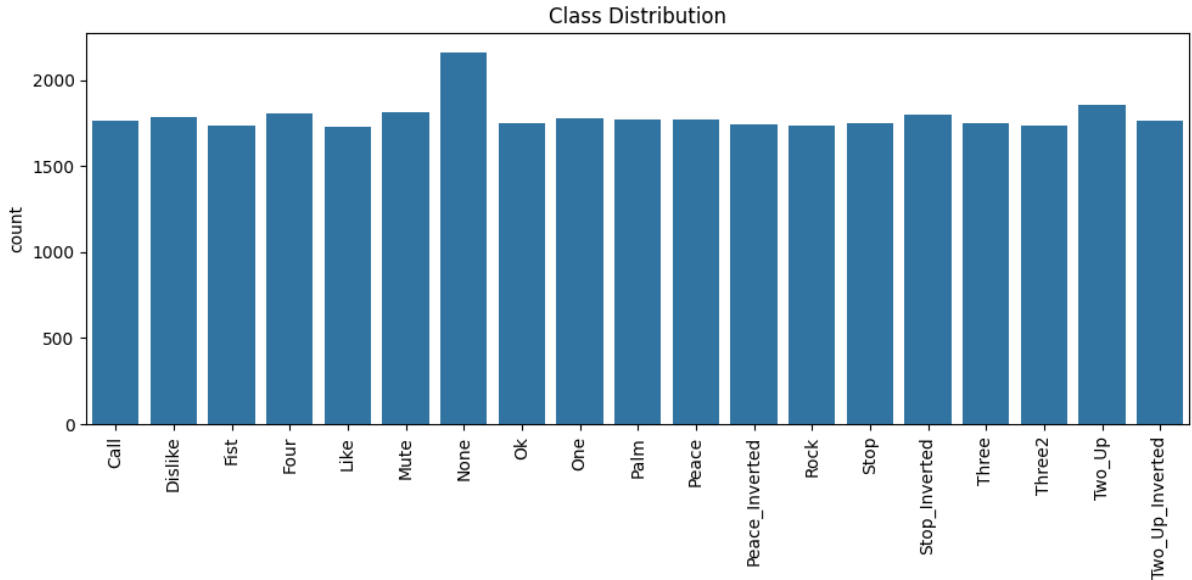


Figure 5.2: Class Distribution graph

The bar chart shows the number of samples available for each gesture class in the dataset.

- Each bar corresponds to a unique gesture label (e.g., Call, Fist, Palm, Two_Up, etc.).
- Most gesture classes are fairly balanced, with sample counts ranging between 1700 and 1900.
- The class None, which represents a "no gesture" class, has the highest frequency with over 2100 samples. This is common in gesture datasets to help the model recognize "non-action" frames.
- Balanced classes help prevent model bias and ensure fair performance across all gestures.

5.3.1 Model Evaluation Metrics

After training the custom gesture recognition model, it was evaluated on a separate test set. The following metrics summarize its performance:

- **Test Accuracy:** 0.7369 The model correctly predicted the gesture label in approximately 73.7% of test cases.
- **Precision:** 0.7787 Out of all gestures the model predicted as a certain class, around 77.9% were actually correct. This reflects the model's ability to avoid false positives.
- **Recall:** 0.7369 The model successfully retrieved 73.7% of all actual gestures. This indicates how well it avoids missing true gestures.
- **F1 Score:** 0.7471 This value suggests that the model achieves a good trade-off between being precise and being comprehensive.

These metrics indicate that the model performs reliably across a diverse set of gestures, though there is still room for improvement, particularly in increasing recall and accuracy through additional data augmentation or model tuning.

5.3.2 Precision–Recall Tradeoff Analysis

To better understand the model's behavior under different classification thresholds, we analyzed the tradeoff between precision and recall. Figure 5.5 illustrates how both metrics evolve as the decision threshold varies from 0 to 1.

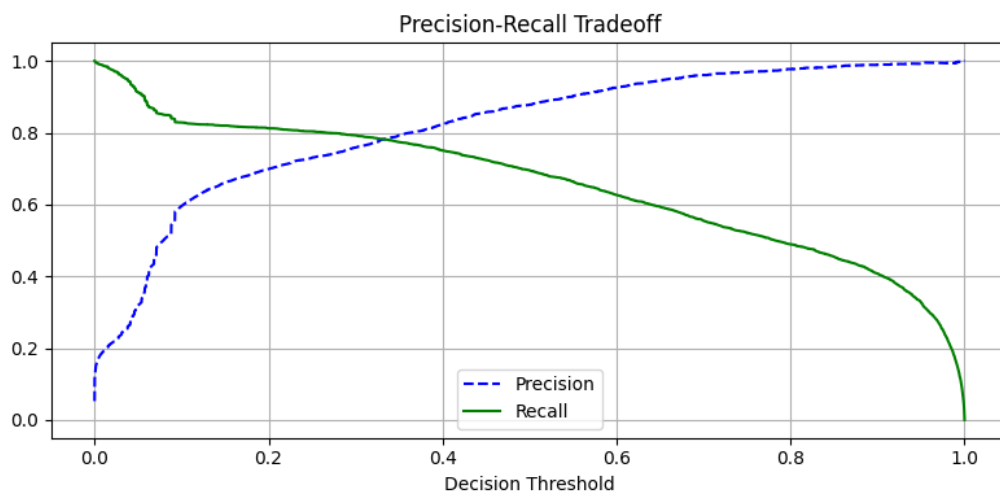


Figure 5.3: Precision–Recall tradeoff curve. Precision improves at higher thresholds, while recall declines.

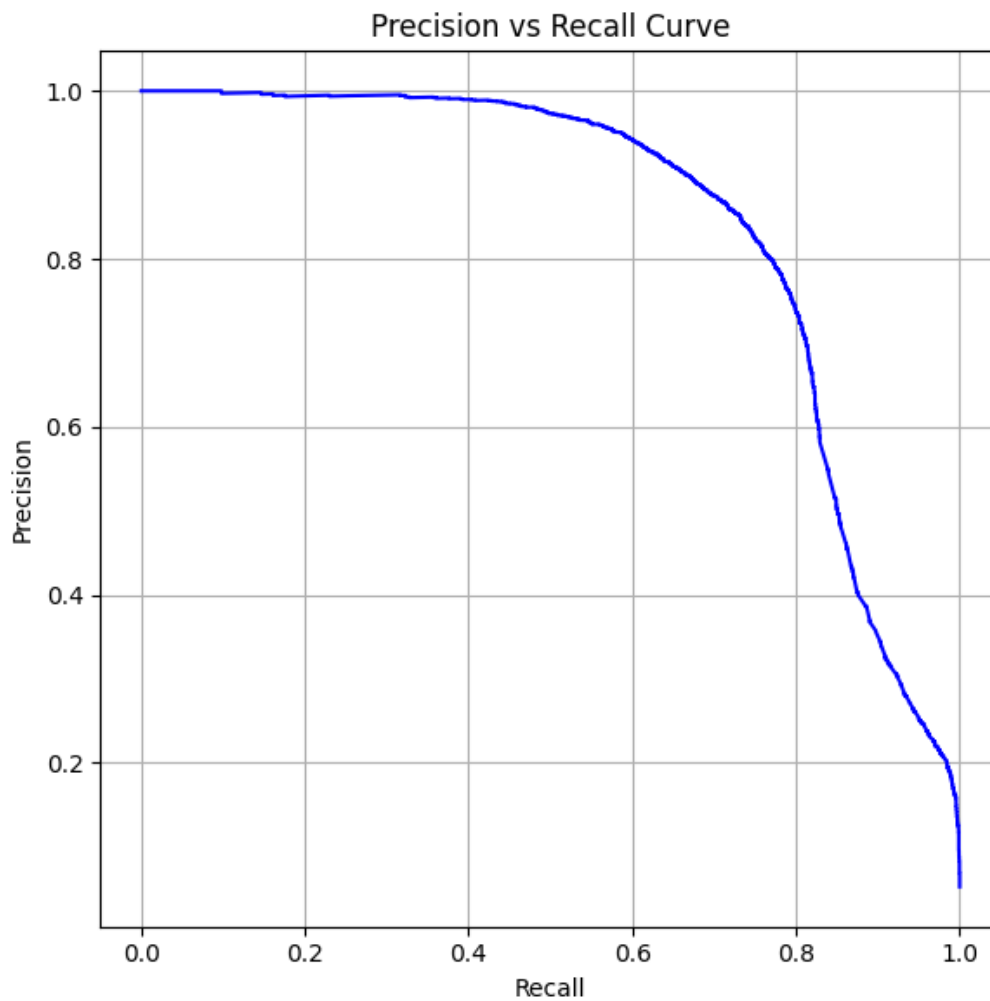


Figure 5.4: Precision vs Recall curve

At lower thresholds, recall is high because the model classifies more inputs as gestures, but precision suffers due to an increase in false positives. As the threshold increases, the model becomes more conservative — precision improves, but recall drops as more true gestures are missed.

5.3.3 Confusion Matrix

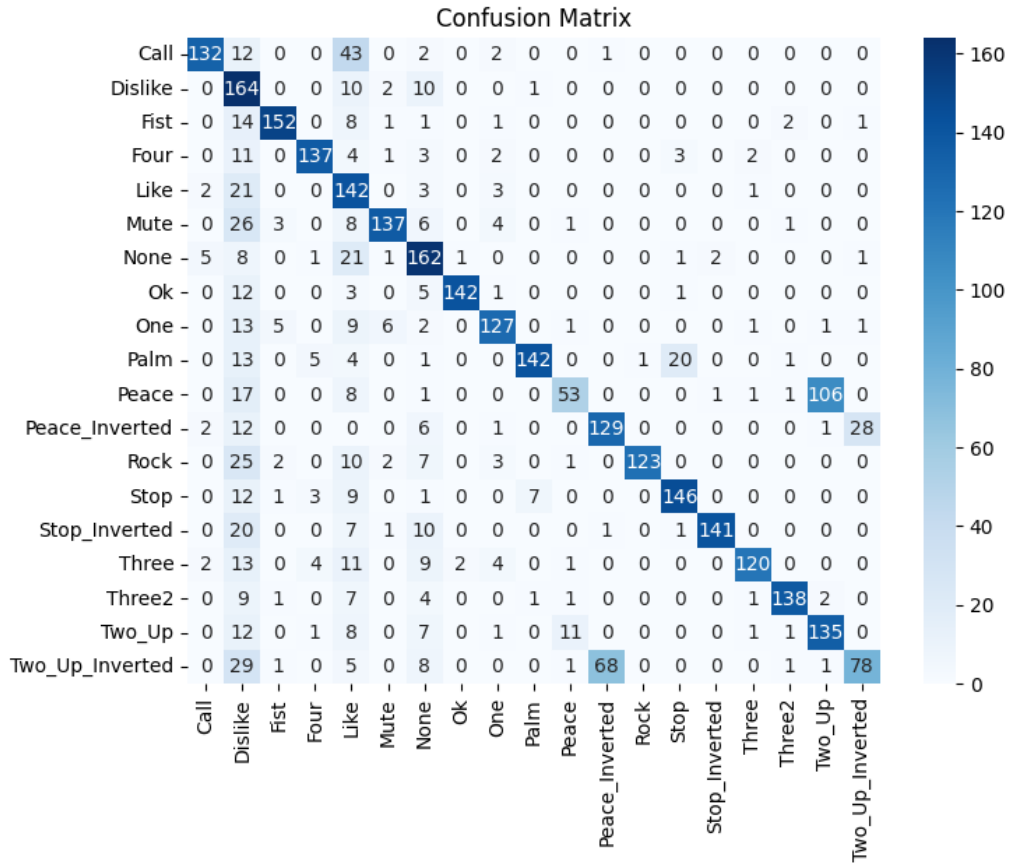


Figure 5.5: Confusion Matrix. The diagonal elements indicate correct predictions, while off-diagonal elements represent misclassifications.

5.3.4 Confusion Matrix Analysis

To further evaluate the performance of the gesture recognition model, a confusion matrix was generated, as shown in the figure above. The matrix provides a detailed view of how well the model is able to correctly classify each gesture and where it tends to make mistakes.

Key Observations

- The model performs well on most gestures, with strong diagonal values for classes like None (162), Stop (146), Palm (142), and Dislike (164), indicating high classification accuracy.
- Some gestures exhibit confusion with visually or structurally similar gestures. For example:
 - Call is sometimes misclassified as None (43 times).
 - Peace_Inverted has significant overlap with Two_Up_Inverted and Peace, possibly due to mirrored hand poses.
 - One and Three have occasional cross-predictions, likely due to similar finger counts.

- The presence of small but consistent off-diagonal errors suggests that the model may benefit from better data augmentation, gesture alignment, or finer-grained landmark features.
- Some gestures like Peace, Peace_Inverted, and Two_Up_Inverted have relatively lower recognition accuracy, which may point to class imbalance or ambiguity in training data.

Conclusion

The confusion matrix reveals that while the model performs well overall, it struggles with certain gestures that are visually similar or exhibit orientation-based variability. Future improvements could include expanding the dataset for ambiguous classes.

5.3.5 Training and Validation Curve Analysis

The training process of the custom gesture recognition model was monitored over 20 epochs using both the training and validation datasets. Figures 5.6 and 5.7 present the corresponding loss and accuracy curves, respectively.

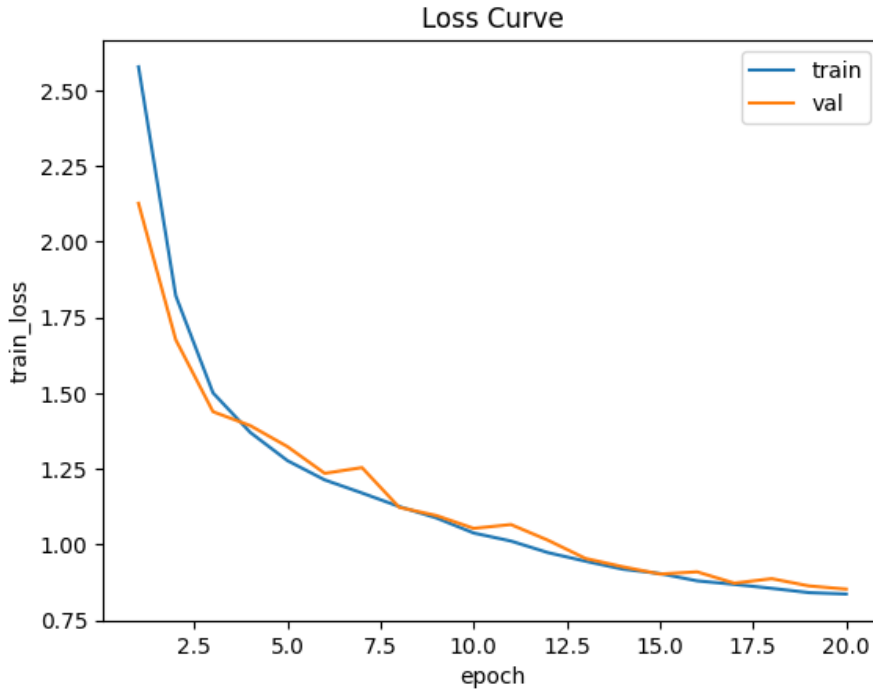


Figure 5.6: Loss curve over 20 epochs for training and validation datasets.

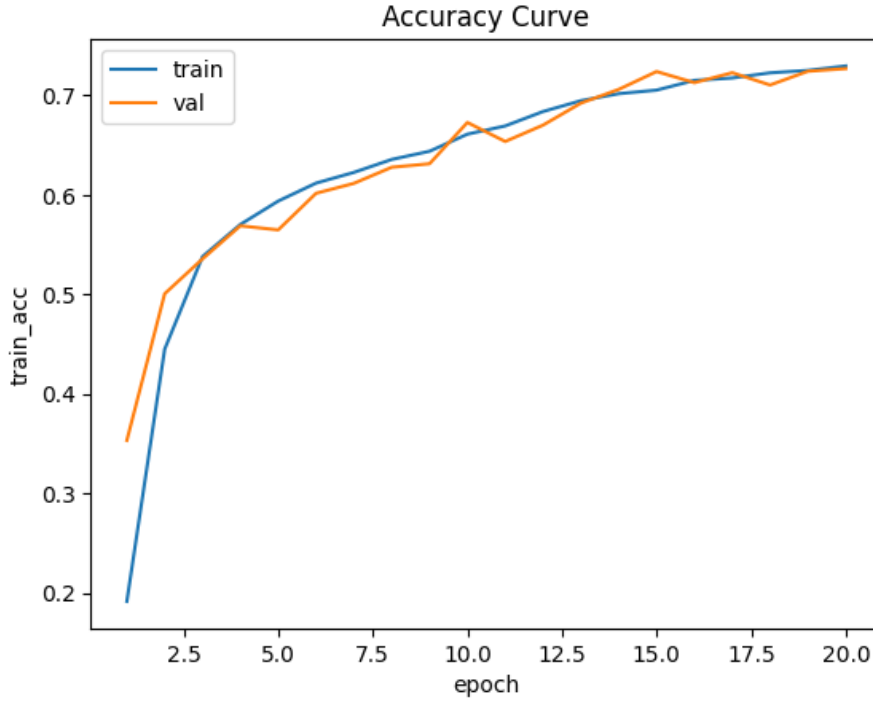


Figure 5.7: Accuracy curve over 20 epochs for training and validation datasets.

Loss Curve Interpretation

The training loss (blue) and validation loss (orange) both show a steady downward trend throughout training. The initial gap between the two curves closes by around epoch 5, and they remain close for the rest of training. This indicates that the model is learning effectively and generalizing well, with no major signs of overfitting.

Accuracy Curve Interpretation

Similarly, the training and validation accuracy curves show consistent improvement over time. Validation accuracy remains close to training accuracy at every epoch, suggesting that the model is not overfitting and that its performance on unseen data closely follows its behavior on the training set. Final validation accuracy approaches 74%, matching the performance metrics reported elsewhere in this report.

5.4 Advantages and Limitations

Advantages

- Easily customizable gesture vocabulary
- Lightweight enough for real-time use

Limitations

- Limited scalability for motion sequences.
- Requires GPU to achieve near-max performances.

5.5 Comparison with Pretrained Mediapipe

Compared to the pretrained Mediapipe approach, this DNN model:

- Performs better on custom or culturally specific gestures
- Requires preprocessing and training but supports flexibility
- Sacrifices some robustness in unusual lighting or poses

5.6 Conclusion

The custom DNN branch balances accuracy and performance, offering a more flexible alternative to rule-based methods. Though it introduces the complexity of dataset management and training, it provides better adaptability for real-world use cases, especially where predefined gesture sets are insufficient.

6 Branch Analysis: Custom Mediapipe Model

6.1 Overview

The `custom_mediapipe_model` branch leverages Google Mediapipe's infrastructure to run a custom-trained gesture classifier.

This approach offers the advantages of using Mediapipe's performance-optimized inference pipeline while still enabling gesture customization based on user-defined data.

Since this branch uses Mediapipe as its base, the only difference between this branch and the main one are the imported model in `gesture_recognizer.py` and the gesture tuple in `flask_client.py`.

This branch was created to experiment with mediapipe-model-maker framework and to add new gestures.

6.2 Custom Dataset

The dataset used is the same as the one in Custom DNN Model branch of the previous chapter.

6.3 Training Script

Listing 6.1: Loading model and training

```
data = gesture_recognizer.Dataset.from_folder(
    dirname=dataset_path,
    hparams=gesture_recognizer.HandDataPreprocessingParams()
)
train_data, rest_data = data.split(0.8)
validation_data, test_data = rest_data.split(0.5)
```

This code loads the dataset located at `dataset_path` by using the `Dataset.from_folder` method. When loading the dataset, the pre-packaged hand detection model from Mediapipe Hands detects the hand landmarks from the images. Any images without detected hands are omitted from the dataset. The resulting dataset will contain the extracted hand landmark positions from each image, rather than images themselves. The dataset is split: 80% for training, 10% for validation, and 10% for testing.

Listing 6.2: Loading model and training

```
hparams = gesture_recognizer.HParams(export_dir="exported_model")
options = gesture_recognizer.GestureRecognizerOptions(hparams=
    ↪ hparams)
model = gesture_recognizer.GestureRecognizer.create(
    train_data=train_data,
    validation_data=validation_data,
    options=options
)
```

This code trains the custom gesture recognizer by using the `create` method and passing in the training data, validation data, model options, and hyperparameters.

6.4 Conclusion

The custom Mediapipe model branch represents an advanced, production-level approach to gesture recognition. It combines the efficiency of Mediapipe's runtime with the adaptability of a custom-trained model. While its setup is technically demanding, it provides a compelling direction for scalable and fast gesture recognition systems.

7 Client-Server Interaction

7.1 Architecture Overview

The system is designed using a client-server model in which gesture recognition and user input occur on the client side, while the server handles execution of system-level commands based on recognized gestures.

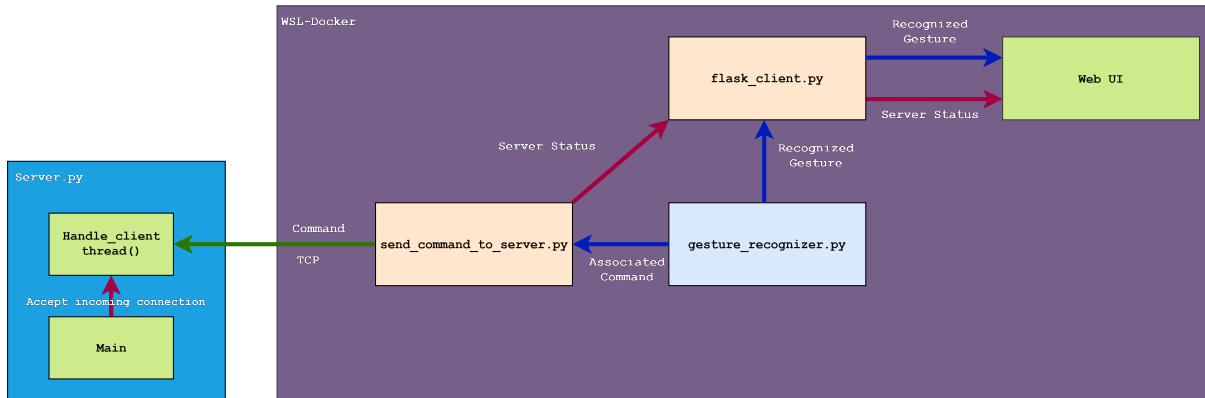


Figure 7.1: Client-Server Communication Flow

7.2 Client Side

The `send_command_to_server.py` module is responsible for sending system commands to the server over a TCP connection. It continuously listens for new commands coming from the gesture recognition process via a multiprocessing queue. Once a gesture is recognized and mapped to a command, this module forwards that command to the server for execution. It also handles connection management — automatically reconnecting if the server becomes unreachable — and updates a shared memory flag to indicate the server's current status. This setup allows seamless communication between the gesture interface and the backend system in a decoupled and reliable way.

Listing 7.1: `send_command_to_server.py`

```
import multiprocessing
import socket
import sys
import signal
import ctypes

# TCP server configuration
SERVER_IP = "host.docker.internal"
SERVER_PORT = 9000

# TCP communication with the command server
def send_command_to_server(gesture_recognizer_to_socket_queue : "
    ↳ multiprocessing.Queue", server_is_running : "ctypes.c_bool"
    ↳ ) -> None:

    def handle_sigterm(signum, frame) -> None:
        sys.exit(0)
```

```

# Create a signal handler for SIGTERM to gracefully close the
    ↪ connection
signal.signal(signal.SIGTERM, handle_sigterm)
s = None # Initialize socket to None to avoid
    ↪ UnboundLocalError in case of exception before
    ↪ connection
while True:
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM
            ↪ ) as s:
            s.connect((SERVER_IP, SERVER_PORT))
            # Set the server_is_running flag to True to
            ↪ signal that the server is running to
            ↪ flask_client.py
            server_is_running.value = True
            while True:
                # Wait for a command from the queue
                command = gesture_recognizer_to_socket_queue.
                    ↪ get()
                s.sendall(command.encode())
    except SystemExit:
        # Handle SystemExit to gracefully exit the process
        if s is not None:
            try:
                s.shutdown(socket.SHUT_RDWR) # Not needed,
                    ↪ but can be used to close the socket
                    ↪ gracefully
            except Exception:
                pass
            raise # Pass the SystemExit exception to exit the
                ↪ process
    except (BrokenPipeError, ConnectionResetError) as e:
        print(f"[ERROR] Lost connection to server: {e}")
        server_is_running.value = False
    except Exception as e:
        print(f"[ERROR] Connection to server failed: {e}")

```

Key concepts:

1. `SERVER_IP`: "host.docker.internal" is a special DNS name that, when used inside a container, refers to the address of the container's host machine.
2. `send_command_to_server()`: since this module is started as a `main.py` subprocess, we need to encapsulate it in a function. To communicate with `flask_client.py` and `gesture_recognizer.py`, `main.py` needs to pass it the following parameters:
 - (a) `gesture_recognizer_to_socket_queue`: this PIPE will be used by this module to pop the recognized gestures by the `gesture_recognizer.py` process.
 - (b) `server_is_running`: this boolean will be set by this module to `TRUE` when the connection begins and to `FALSE` when it ends (this occurs when `BrokenPipeError` or `ConnectionResetError` are raised). This boolean will be read by `flask_client.py`, which will send it to the web interface (in order to show the user the current server status).
3. In this module too, `SIGTERM` is handled gracefully, so that the TCP connection can end correctly.

7.3 Server Side

The `server.py` module implements a custom TCP server that listens for gesture-triggered commands from the client and executes system-level actions in response. It is designed to run indefinitely and handle incoming connections using multithreading, ensuring that multiple clients can interact with it simultaneously.

Once a connection is established, the server waits for command strings (such as "Volume Up" or "Open Calculator") sent by the gesture recognition system. These commands are parsed and matched against predefined actions, which include:

- Adjusting system volume (`volume_up()` and `volume_down()`)
- Simulating keyboard shortcuts (`simulate_print_screen()`)
- Controlling media playback (`simulate_media_play_pause()`)
- Opening applications like Calculator or Task Manager (`open_calculator()`, `open_task_manager()`)
- Performing mouse scroll actions (`scroll_mouse()`)

The server uses Windows-specific libraries such as:

- `ctypes` to simulate keyboard and mouse events
- `pycaw` to manage system audio
- `psutil` to check running processes

Threading is used to manage client sessions, ensuring responsive behavior even with multiple active connections. To prevent repeated execution of the same command, the server checks if an app like Calculator or Task Manager is already open before launching it again.

The server is designed with robustness in mind: it handles connection timeouts, decoding errors, and graceful shutdown via COM resource cleanup. It runs in a loop, restarting listening after each handled connection.

Listing 7.2: *server.py*

```
# TCP Server
def handle_client(conn, addr) -> None:
    pythoncom.CoInitialize()
    try:
        with conn:
            while True:
                # Set a timeout for receiving data to avoid
                # ↪ blocking indefinitely
                data = conn.recv(1024)
                try:
                    # Split by '|' into an array and take the
                    # ↪ first element.
                    # To avoid issues where tcp buffers commands
                    # ↪ and sends them in chunks
                    command = data.decode('utf-8').strip().split(
                        ↪ '|'')[0]
                except UnicodeDecodeError:
                    continue
```

```

        # If command is empty, skip processing to speed
        ↪ up the loop
        if not command:
            continue

    # Process the command
    if command == "Volume_Up":
        response = volume_up()
    elif command == "Volume_Down":
        response = volume_down()
    elif command == "AltTab":
        simulate_alt_tab()
        response = "Alt+Tab_sent"
    elif command == "PlayPause":
        simulate_media_play_pause()
        response = "Media_play/pause_triggered"
    elif command == "Open_Calculator":
        if calculator_already_running():
            continue
        open_calculator()
        response = "Calculator_opened"
    elif command == "Screenshot":
        simulate_print_screen()
        response = "Screenshot_key_(Print_Screen)_
        ↪ sent"
    elif command == "Scroll_Up":
        scroll_mouse(120)
        response = "Mouse_scrolled_up"
    elif command == "Scroll_Down":
        scroll_mouse(-120)
        response = "Mouse_scrolled_down"
    elif command == "Task_Manager":
        if task_manager_already_running():
            continue
        open_task_manager()
        response = "Task_Manager_opened"
    else:
        response = f"Unknown_command:{command}"

    print(f"[RESPONSE]_{response}")

finally:
    # Deinitialize COM to clean up resources
    pythoncom.CoUninitialize()

def main():
    # Initialize the server socket
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        # Set socket options to allow address reuse
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        # Bind the socket to the specified host and port
        s.bind((HOST, PORT))
        # Start listening for incoming connections
        s.listen()

```

```

# Set a timeout for accepting connections to avoid
    ↪ blocking indefinitely and accept Ctrl+C
s.settimeout(1.0)
while True:
    try:
        # Accept a new client connection
        conn, addr = s.accept()
        # Create a new thread to handle the client
        ↪ connection
        # Use daemon threads so they will exit when the
        ↪ main thread exits
        thread = threading.Thread(target=handle_client,
        ↪ args=(conn, addr), daemon=True)
        thread.start()
    except socket.timeout:
        # If no connection is accepted within the timeout
        ↪ , continue to check for new connections
        continue
    except Exception as e:
        # Catch any other exceptions (like
        ↪ KeyboardInterrupt, launched with CTRL+C)
        ↪ and print an error message
        print(f"[ERROR] {e}")
        break
    print("[STOP] Server arrested")

if __name__ == '__main__':
    main()

```

7.4 Conclusion

The client-server interaction is fundamental to the gesture control system. It ensures clean separation of UI logic from low-level command execution, making the system scalable, modular, and suitable for deployment in real-world scenarios.

8 Frontend Interface

8.1 Overview

The frontend interface is built using the Flask web framework and serves a single-page application (SPA) via HTML and JavaScript. It is located in the `client/templates/` and `client/static/` directories and served by `flask_client.py`.

The interface provides:

- A start button to trigger gesture recognition.
- A stop button to stop gesture recognition.
- A save button to save the selected gesture to command configuration.
- A select element that lets the user select a previously saved configuration.
- A load button to apply the gesture to commands bindings which are present in the form at that moment.
- Real-time feedback via AJAX.
- Basic UI structure using HTML/CSS.

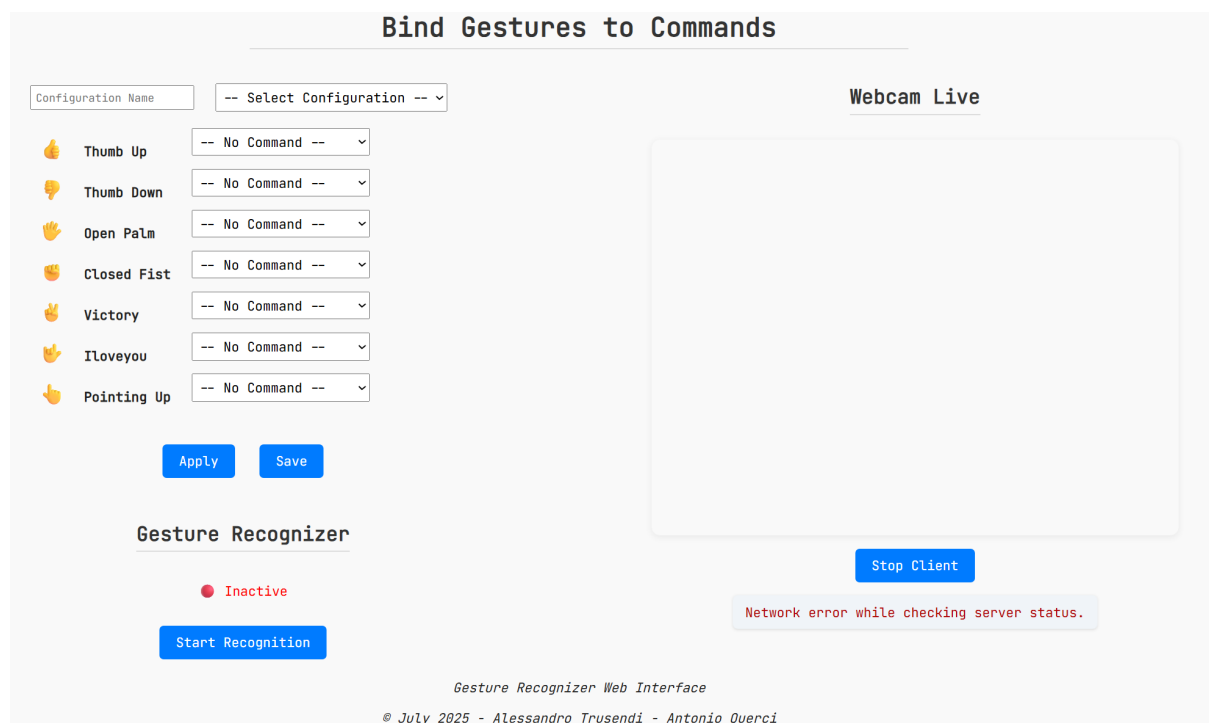


Figure 8.1: Placeholder: Frontend UI Layout

8.2 HTML Template

The main user interface is rendered using Jinja2 templates. As you can see in `index.html` file, the Web UI is structured in the follow way:

- Generate a dynamic form based on gestures and commands passed as arguments in the `render_template` function in `flask_client.py`.

- Allow the user to save and, later, load configurations.
- Let the user start and stop the gesture recognition.
- Display webcam feed and recognized gesture.
- Stop the client.

8.3 JavaScript Logic

AJAX calls are used to interact with backend Flask endpoints asynchronously. This avoids full page reloads and provides a responsive interface. POST requests are only used to apply and save the selected gesture to command configuration. All the others HTTP request use the GET method.

8.4 Flask Routing Logic

In `flask_client.py`, routes are defined to serve the HTML page and receive AJAX commands:

8.5 User Workflow

The interaction process is as follows:

1. User loads the web interface
2. Clicks the “Start Recognizer” button
3. JavaScript sends an AJAX request
4. Flask spawns the recognizer as a subprocess
5. The recognizer begins webcam capture and gesture classification
6. Recognized gestures are displayed or trigger backend commands

8.6 Future UI Improvements

Use of relative units in css.

9 Docker and Development Container Integration

To ensure portability, consistent environments, and an easier development setup, this project makes use of Docker and Visual Studio Code's Dev Container feature. These tools provide isolated, reproducible environments that simplify deployment, testing, and collaboration across different machines.

9.1 Motivation

Gesture recognition applications involve several platform-specific dependencies, such as OpenCV, Mediapipe, and access to webcam devices. Setting up such an environment manually can be error-prone due to variations in Python versions, system libraries, or hardware access permissions. Docker and Dev Containers solve this problem by encapsulating the development environment within a self-contained, version-controlled container.

9.2 Dockerfile Benefits

The provided `Dockerfile` builds a minimal and compatible image for running the client:

- Installs required system libraries for OpenCV and webcam usage.
- Sets up the working directory and mounts a volume for saving gesture-command configurations.
- Installs Python packages from a locked requirements file for dependency stability.
- Runs the application as a non-root user with access to video devices.

This makes the container suitable for running the web interface, streaming webcam input, and interacting with gesture recognition models — all without polluting the host system.

9.3 Dev Container Configuration

The `devcontainer.json` file enables Visual Studio Code to automatically open the project in a fully-configured containerized environment. It:

- Mounts the host webcam inside the container using `--device=/dev/video0`.
- Shares the host's display socket (`/tmp/.X11-unix`) to allow GUI applications and OpenCV visualizations.
- Mounts the project directory and config folder for persistent development.
- Installs useful extensions like Python, Pylance, Jupyter, and Live Share for a productive coding experience.

• Dev Container Configuration

The `devcontainer.json` file enables Visual Studio Code to automatically open the project in a fully-configured containerized environment. It:

- Mounts the host webcam inside the container using `--device=/dev/video0`, allowing OpenCV to access camera input.
- Shares the host's display socket (`/tmp/.X11-unix`) so GUI elements (like OpenCV windows) can render on the host machine.
- Mounts the project's `configs` folder into the container to persist user-defined gesture-command configurations.
- Installs helpful development extensions such as Python, Pylance, Jupyter, and Live Share to improve coding efficiency.
- **Forwards port 9000** from the container to the host, which is used for TCP communication between the gesture client and the server. This ensures that services running inside the container (e.g., the command server) are accessible externally, including from the gesture recognizer or test clients running on the host.

This setup ensures that all developers — regardless of OS — can start coding immediately with the correct Python version, tools, and system permissions.

10 Discussion and Future Work

10.1 Summary of Achievements

This project demonstrated a complete, modular gesture recognition system using three distinct modeling strategies. Key accomplishments include:

- Successful integration of real-time hand tracking using Mediapipe.
- Implementation of three gesture classification branches:
 - Pretrained Mediapipe (rule-based)
 - Custom DNN with scikit-learn
 - Custom TFLite model embedded into Mediapipe
- Development of a Flask-based client interface with asynchronous controls.
- Command execution via client-server messaging.

10.2 Technical Challenges

Challenges arose during development:

- Webcam connection to Docker through WSL
- Docker to Windows host connection

AI training, MediaPipe API, data collection, system architecture and AJAX Web UI design were hard tasks too, but they were solved without that much trouble.

10.3 Strengths of the System

- **Modular design:** The system is well-structured and separates frontend, backend, and recognition logic.
- **Multiple models:** Easy to switch between different branches or recognition approaches.
- **Real-time performance:** All models perform well enough to be used in interactive systems.
- **Low-cost deployment:** The use of common libraries and a browser-based UI makes this system highly portable.

10.4 Future Work

1. Multi-Hand and Multi-User Support

Future extensions should include support for multiple simultaneous hand detections, enabling richer gestures (e.g., two-hand commands, collaborative control).

2. Web-Based Webcam Streaming

Integrate live webcam streaming directly into the frontend UI using `flask-video-stream` or WebRTC, giving users real-time visual feedback.

3. Gesture Confidence and Debugging UI

Display gesture predictions with confidence scores, allowing users to visualize model behavior and debug false positives.

4. Online Training and Gesture Customization

Enable users to define new gestures from the browser interface and train them using few-shot learning techniques.

5. Deployment to Edge Devices

Deploy the system to Raspberry Pi or NVIDIA Jetson Nano platforms for use in smart environments or robotics.

Bibliography

- https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer/pythonlive-stream